CLOUD COMPUTING — TA-II

Design and Deployment of an Event-Driven Serverless File Processing Architecture Using AWS CloudFormation

---

Name: *Neel Wankhade*
Roll No.: *C-11*
Semester: *VII*
Department: *Computer Science and Engineering*

---

1. Aim

To design and deploy an event-driven serverless file-processing architecture using AWS CloudFormation. When a file is uploaded to an Amazon S3 bucket, an S3 event notification triggers an AWS Lambda function configured in a private subnet. The function processes the uploaded object, stores the generated metadata in S3, and sends a notification through Amazon Simple Notification Service (SNS).

2. Objectives

- To provision AWS resources using AWS CloudFormation.
- To configure an Amazon S3 bucket for file uploads and processed output storage.
- To configure S3 event notifications to invoke an AWS Lambda function automatically.
- To execute the Lambda function within a private subnet of a VPC.
- To use Amazon SNS to send email notifications after successful processing.
- To verify the workflow using S3 objects, Lambda execution logs, and email notifications.

3. Software and Services Required

- AWS Management Console
- AWS CloudFormation
- Amazon Simple Storage Service (S3)
- AWS Lambda
- Amazon Simple Notification Service (SNS)
- Amazon Virtual Private Cloud (VPC)
- Internet Gateway and NAT Gateway
- AWS Identity and Access Management (IAM)

- Amazon CloudWatch
- Microsoft Word for report preparation

4. Architecture Overview

The architecture follows an event-driven serverless model. Amazon S3 acts as the source of file-upload events. When a file is uploaded under the configured uploads/ prefix, S3 sends an object-created event that invokes the Lambda processing function.

The Lambda function reads the uploaded object, generates JSON metadata, and stores the output under the processed/ prefix in the S3 bucket. It also publishes a summary of the processing operation to an SNS topic, which delivers an email notification to the confirmed subscriber.

The Lambda function is configured in a private subnet within a VPC. The VPC also contains a public subnet, an Internet Gateway, a NAT Gateway, and associated route tables.

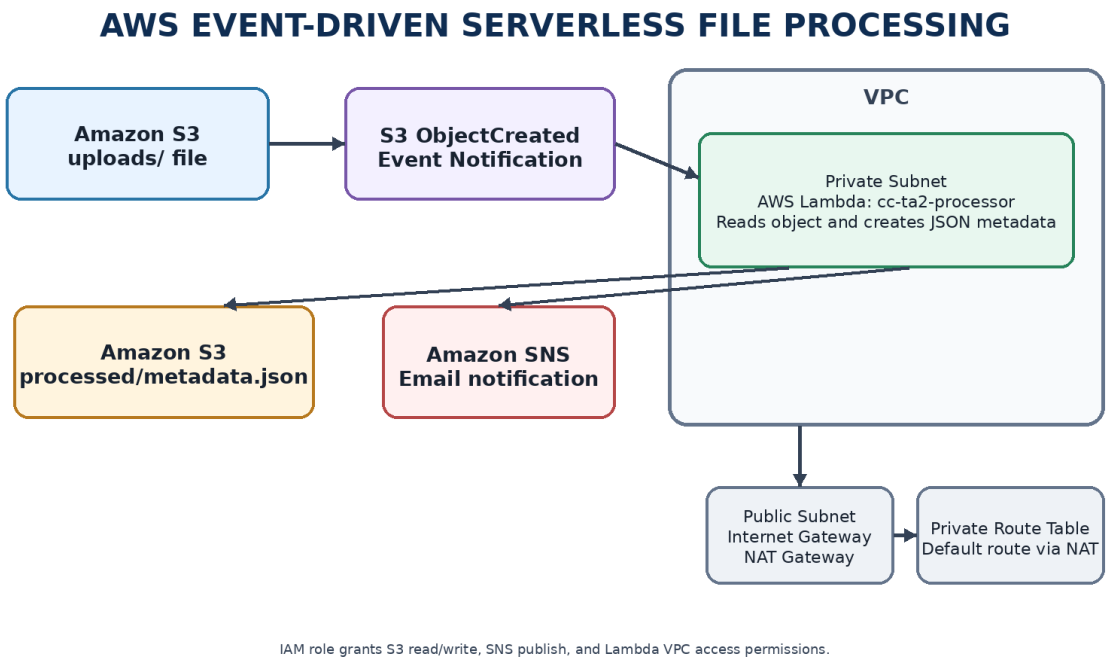


Figure 1: Architecture of the event-driven serverless file-processing system.

5. AWS Resources Used

5.1 AWS CloudFormation

AWS CloudFormation was used to define and provision the infrastructure through a YAML template. The stack was named cc-ta2. It manages the resources required for the file-processing workflow.

5.2 Amazon S3

Amazon S3 provides object storage for incoming files and processed output. The bucket is configured with server-side encryption using S3-managed keys (SSE-S3) and public access blocking.

The uploads/ prefix is used for incoming files, while the processed/ prefix stores the generated JSON metadata.

5.3 AWS Lambda

AWS Lambda provides serverless execution for the file-processing logic. The cc-ta2-processor function is invoked automatically when a matching S3 object-created event occurs.

The function reads the uploaded object, generates metadata in JSON format, stores the metadata in S3, and publishes a processing summary to SNS.

5.4 Amazon SNS

Amazon Simple Notification Service distributes notifications to subscribers. An email subscription was configured and confirmed so that processing notifications could be delivered to the selected email address.

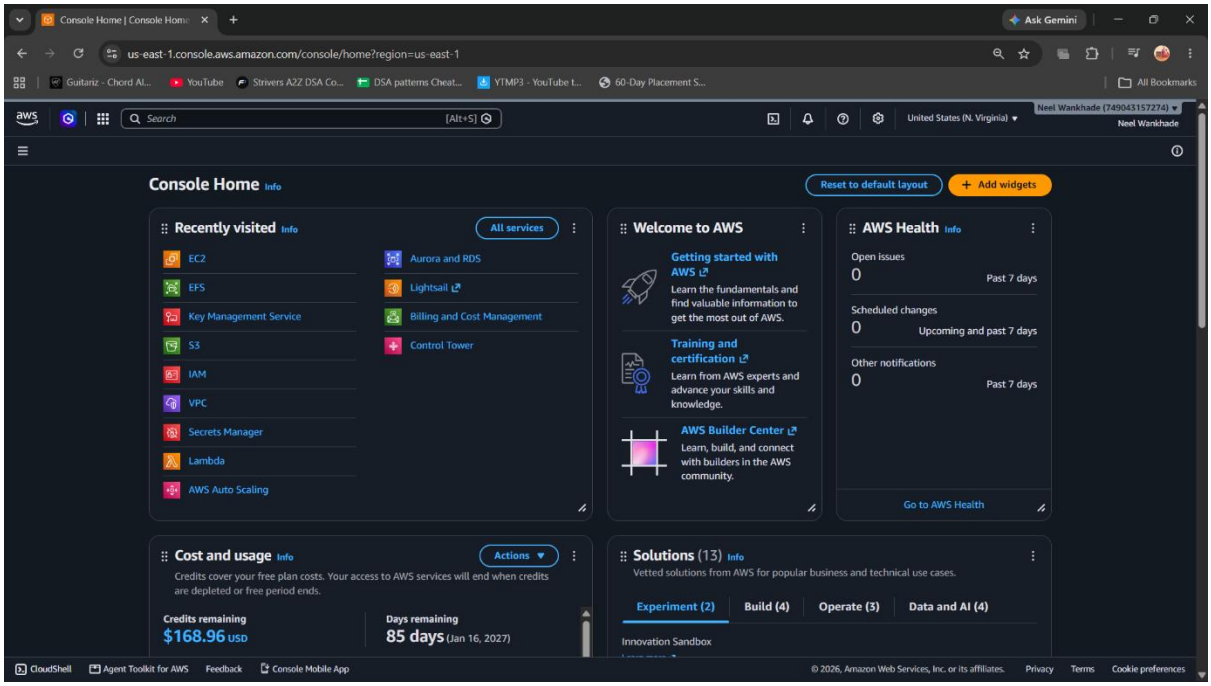
5.5 Amazon VPC and Networking

A VPC was configured with DNS support and DNS hostnames enabled. It includes a public subnet and a private subnet, along with an Internet Gateway, NAT Gateway, route tables, and subnet associations.

The public subnet provides the network path required for the NAT Gateway, while the private subnet hosts the Lambda function. The private route table directs outbound traffic through the NAT Gateway.

5.6 AWS IAM

IAM roles and permissions allow the Lambda function to access the required AWS services. The execution role provides permissions for VPC networking, reading uploaded S3 objects, writing processed metadata, and publishing messages to SNS.

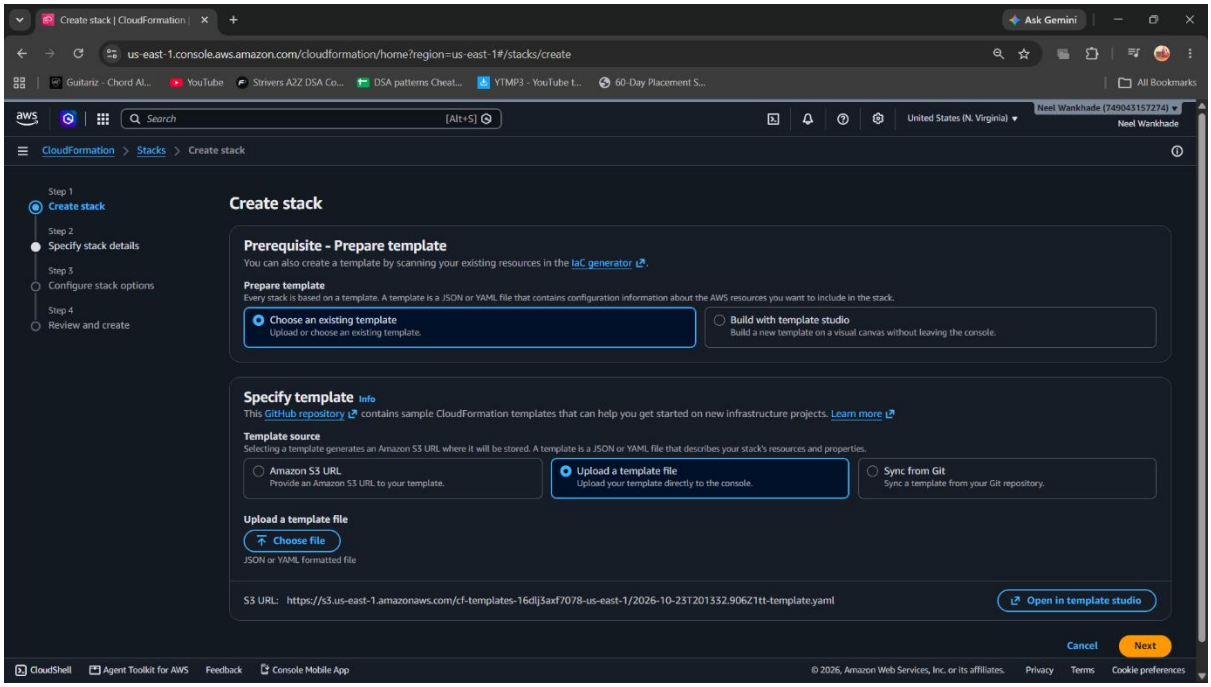


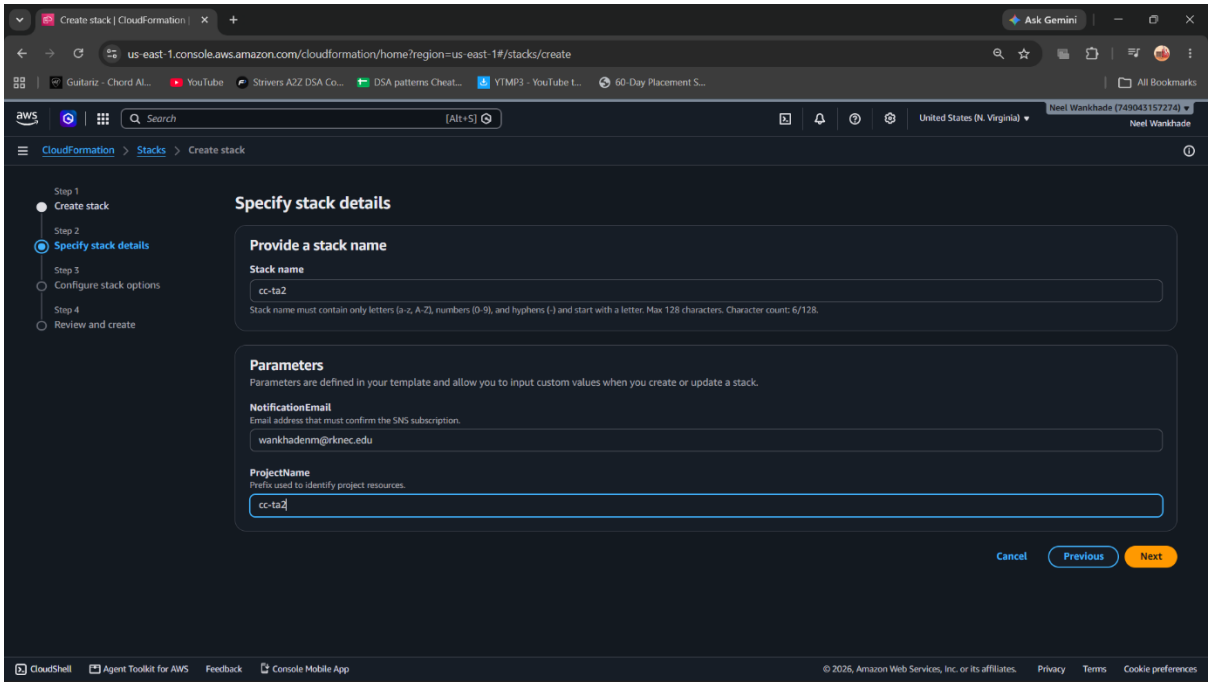
6. Procedure

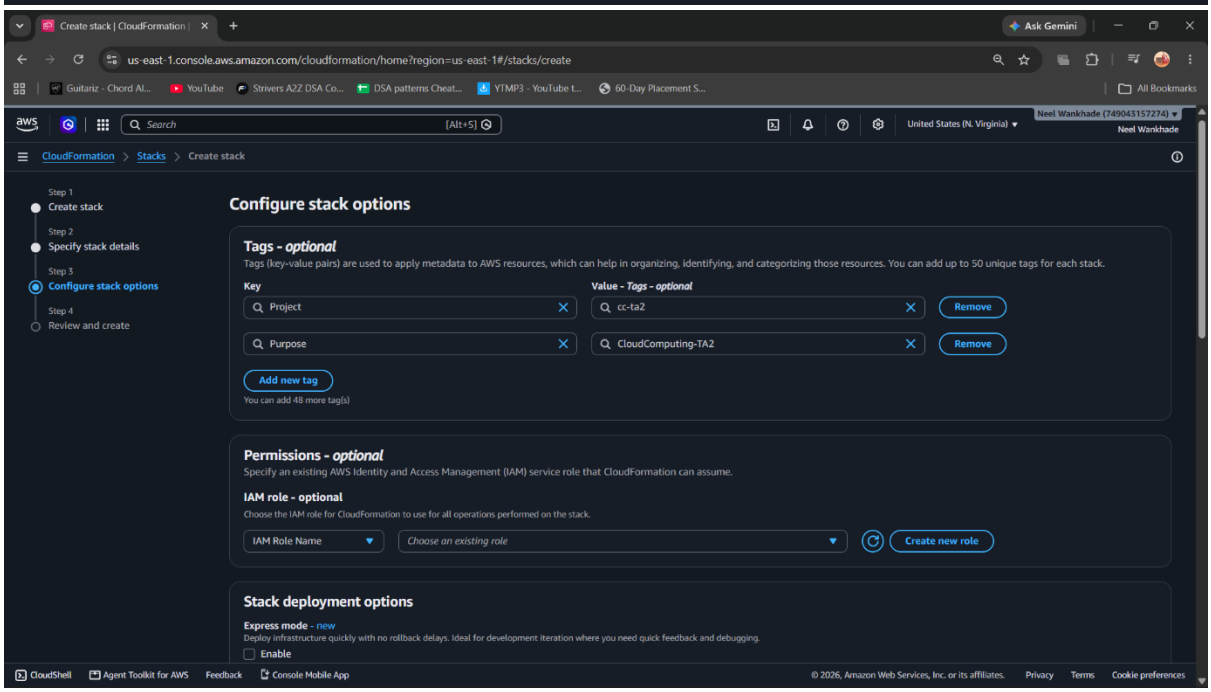
Step 1: Create the CloudFormation Stack

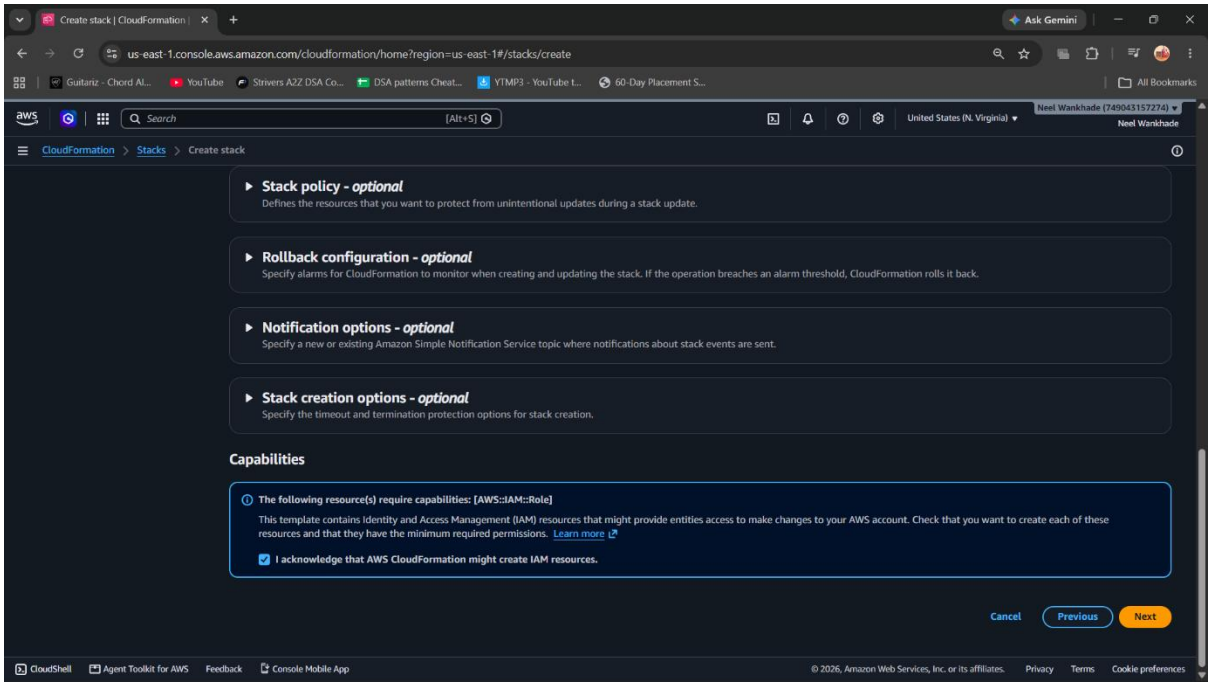
The AWS Management Console was opened in the US East (N. Virginia) region (us-east-1). In CloudFormation, the option to create a new stack was selected, and the template.yaml file was uploaded.

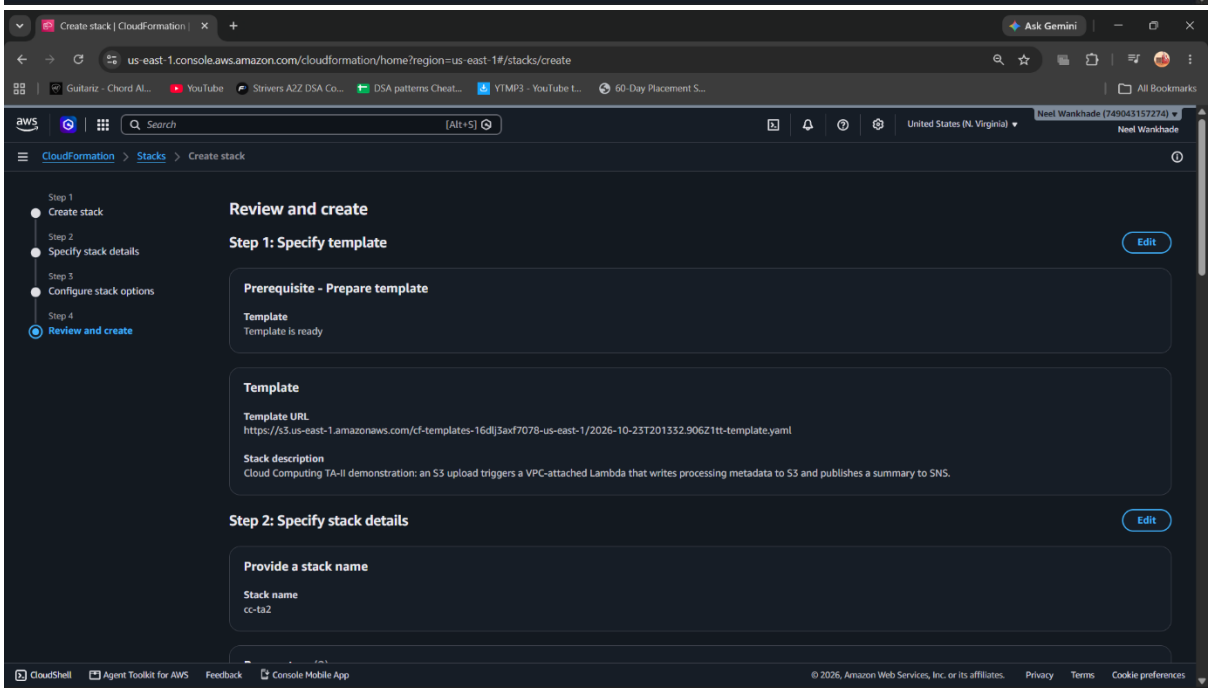
The stack name was set to cc-ta2, and the required parameters, including the notification email address, were provided. The configuration was reviewed and the stack creation was submitted.




Create stack | CloudFormation
Ask Gemini
us-east-1.console.aws.amazon.com/cloudformation/home?region=us-east-1#/stacks/create
Search
[Alt+S]
United States (N. Virginia)
Neel Wankhade (749043157274)
Neel Wankhade
CloudFormation > Stacks > Create stack
Step 1
Create stack
Step 2
Specify stack details
Step 3
Configure stack options
Step 4
Review and create
Specify stack details
Provide a stack name
Stack name
cc-ta2
Stack name must contain only letters (a-z, A-Z), numbers (0-9), and hyphens (-) and start with a letter. Max 128 characters. Character count: 6/128.
Parameters
Parameters are defined in your template and allow you to input custom values when you create or update a stack.
NotificationEmail
Email address that must confirm the SNS subscription.
wankhadenm@rknec.edu
ProjectName
Prefix used to identify project resources.
cc-ta2
Cancel
Previous
Next
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Create stack | CloudFormation
Ask Gemini
us-east-1.console.aws.amazon.com/cloudformation/home?region=us-east-1#/stacks/create
Search
[Alt+S]
United States (N. Virginia)
Neel Wankhade (749043157274)
Neel Wankhade
CloudFormation > Stacks > Create stack
Step 1
Create stack
Step 2
Specify stack details
Step 3
Configure stack options
Step 4
Review and create
Configure stack options
Tags - optional
Tags (key-value pairs) are used to apply metadata to AWS resources, which can help in organizing, identifying, and categorizing those resources. You can add up to 50 unique tags for each stack.
Key
Value - Tags - optional
Project
cc-ta2
Remove
Purpose
CloudComputing-TA2
Remove
Add new tag
You can add 48 more tag(s)
Permissions - optional
Specify an existing AWS Identity and Access Management (IAM) service role that CloudFormation can assume.
IAM role - optional
Choose the IAM role for CloudFormation to use for all operations performed on the stack.
IAM Role Name
Choose an existing role
Create new role
Stack deployment options
Express mode - new
Deploy infrastructure quickly with no rollback delays. Ideal for development iteration where you need quick feedback and debugging.
Enable
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Create stack | CloudFormation
Ask Gemini
us-east-1.console.aws.amazon.com/cloudformation/home?region=us-east-1#/stacks/create
Search
[Alt+S]
United States (N. Virginia)
Neel Wankhade (749043157274)
Neel Wankhade
CloudFormation
Stacks
Create stack
Stack policy - optional
Defines the resources that you want to protect from unintentional updates during a stack update.
Rollback configuration - optional
Specify alarms for CloudFormation to monitor when creating and updating the stack. If the operation breaches an alarm threshold, CloudFormation rolls it back.
Notification options - optional
Specify a new or existing Amazon Simple Notification Service topic where notifications about stack events are sent.
Stack creation options - optional
Specify the timeout and termination protection options for stack creation.
Capabilities
The following resource(s) require capabilities: [AWS::IAM::Role]
This template contains Identity and Access Management (IAM) resources that might provide entities access to make changes to your AWS account. Check that you want to create each of these resources and that they have the minimum required permissions. Learn more
I acknowledge that AWS CloudFormation might create IAM resources.
Cancel
Previous
Next
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Create stack | CloudFormation
Ask Gemini
us-east-1.console.aws.amazon.com/cloudformation/home?region=us-east-1#/stacks/create
Search
[Alt+S]
United States (N. Virginia)
Neel Wankhade (749043157274)
Neel Wankhade
CloudFormation
Stacks
Create stack
Step 1
Create stack
Step 2
Specify stack details
Step 3
Configure stack options
Step 4
Review and create
Review and create
Step 1: Specify template
Edit
Prerequisite - Prepare template
Template
Template is ready
Template
Template URL
https://s3.us-east-1.amazonaws.com/cf-templates-16dlj3axf7078-us-east-1/2026-10-23T201332.906Z1tt-template.yaml
Stack description
Cloud Computing TA-II demonstration: an S3 upload triggers a VPC-attached Lambda that writes processing metadata to S3 and publishes a summary to SNS.
Step 2: Specify stack details
Edit
Provide a stack name
Stack name
cc-ta2
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

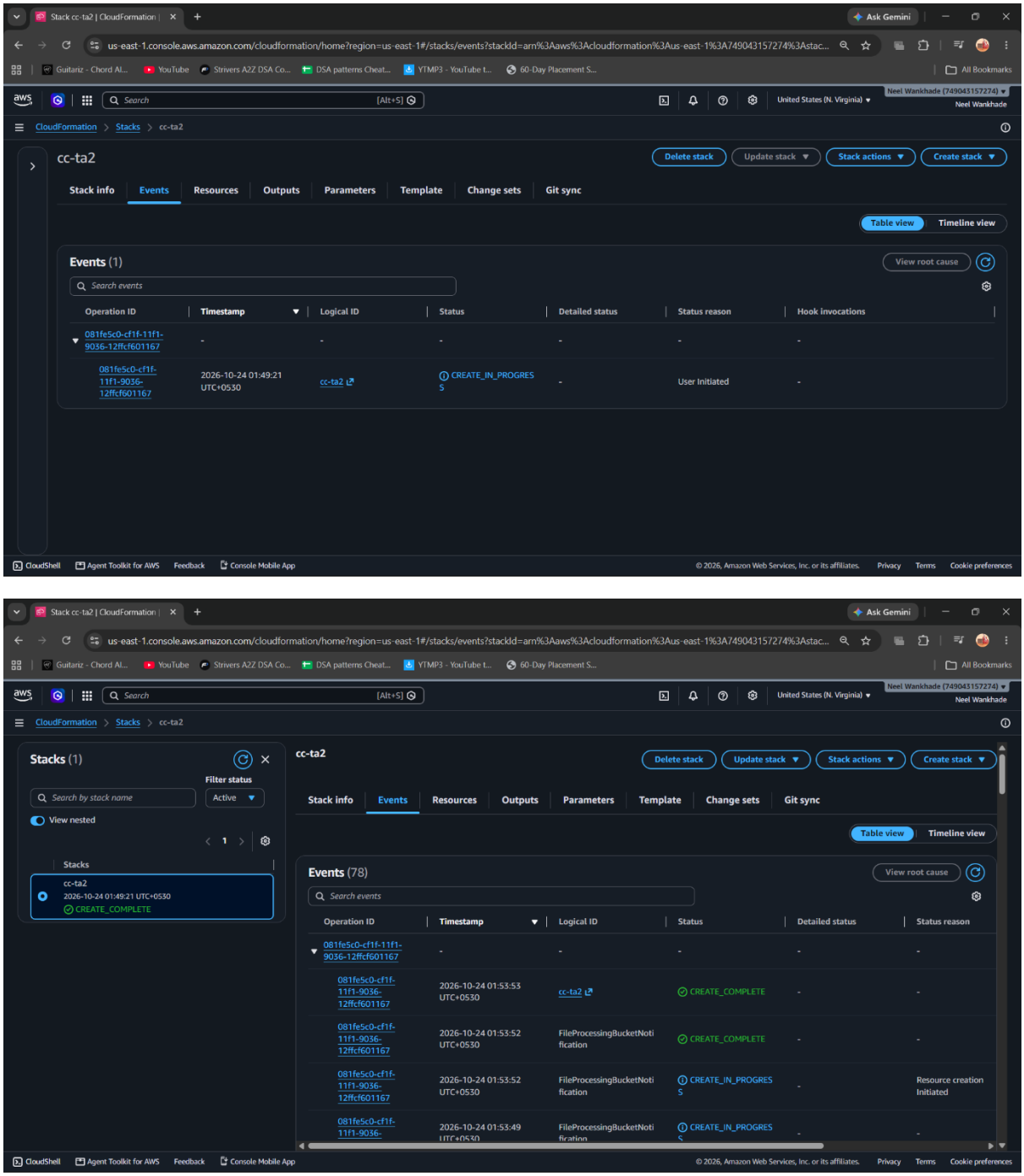


Figure 2: Successful creation of the CloudFormation stack.

Step 2: Configure the SNS Email Subscription

An SNS topic was created as part of the CloudFormation deployment. An email subscription was configured using the selected email address. The subscription confirmation message was opened, and the confirmation link was selected.

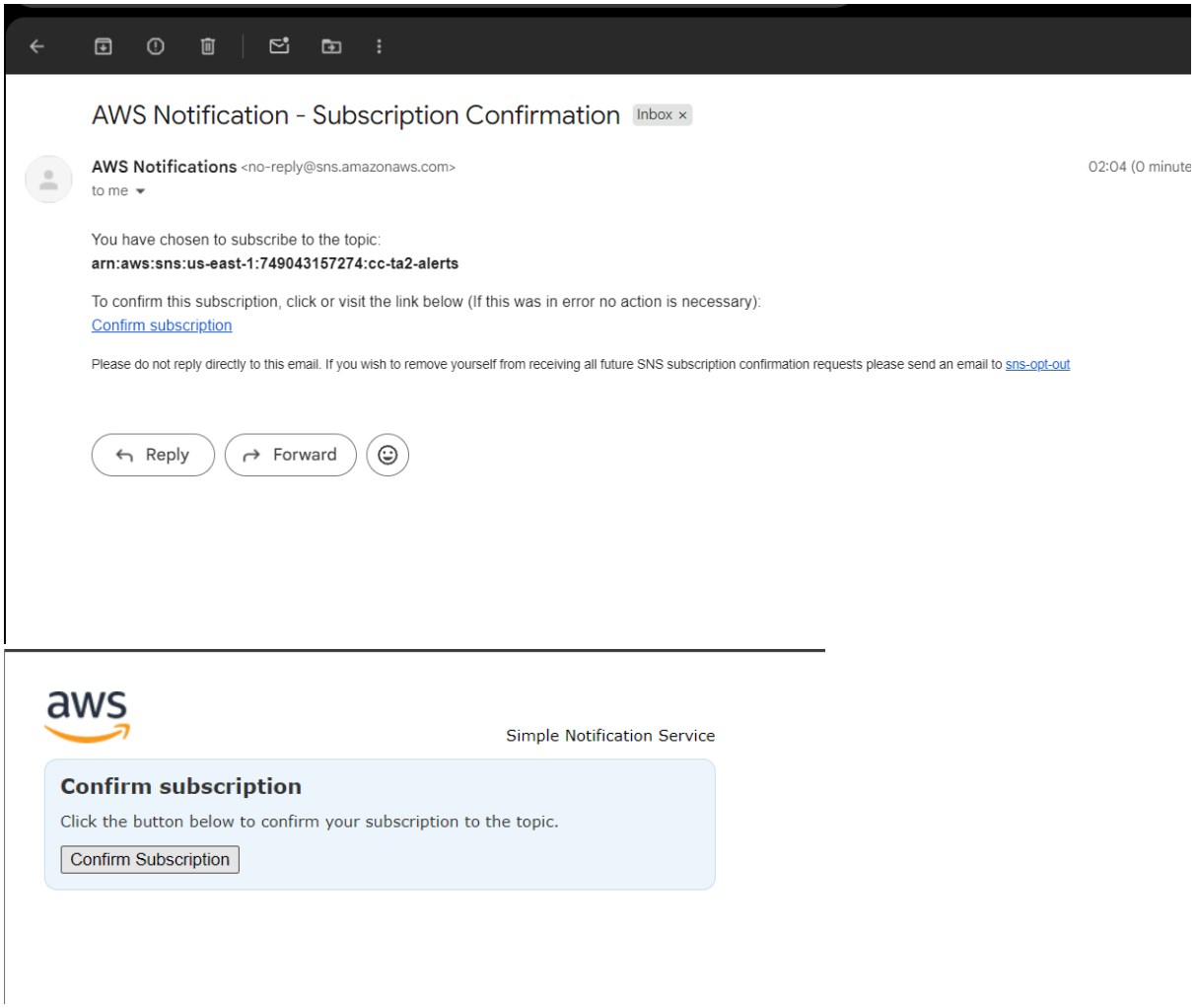


Figure 3: Confirmation of the SNS email subscription.

Step 3: Upload a Test File to Amazon S3

The S3 bucket created by CloudFormation was opened. A test file named test.txt was uploaded under the uploads/ prefix.

The bucket's event notification configuration was set to respond to object-created events for files under this prefix.

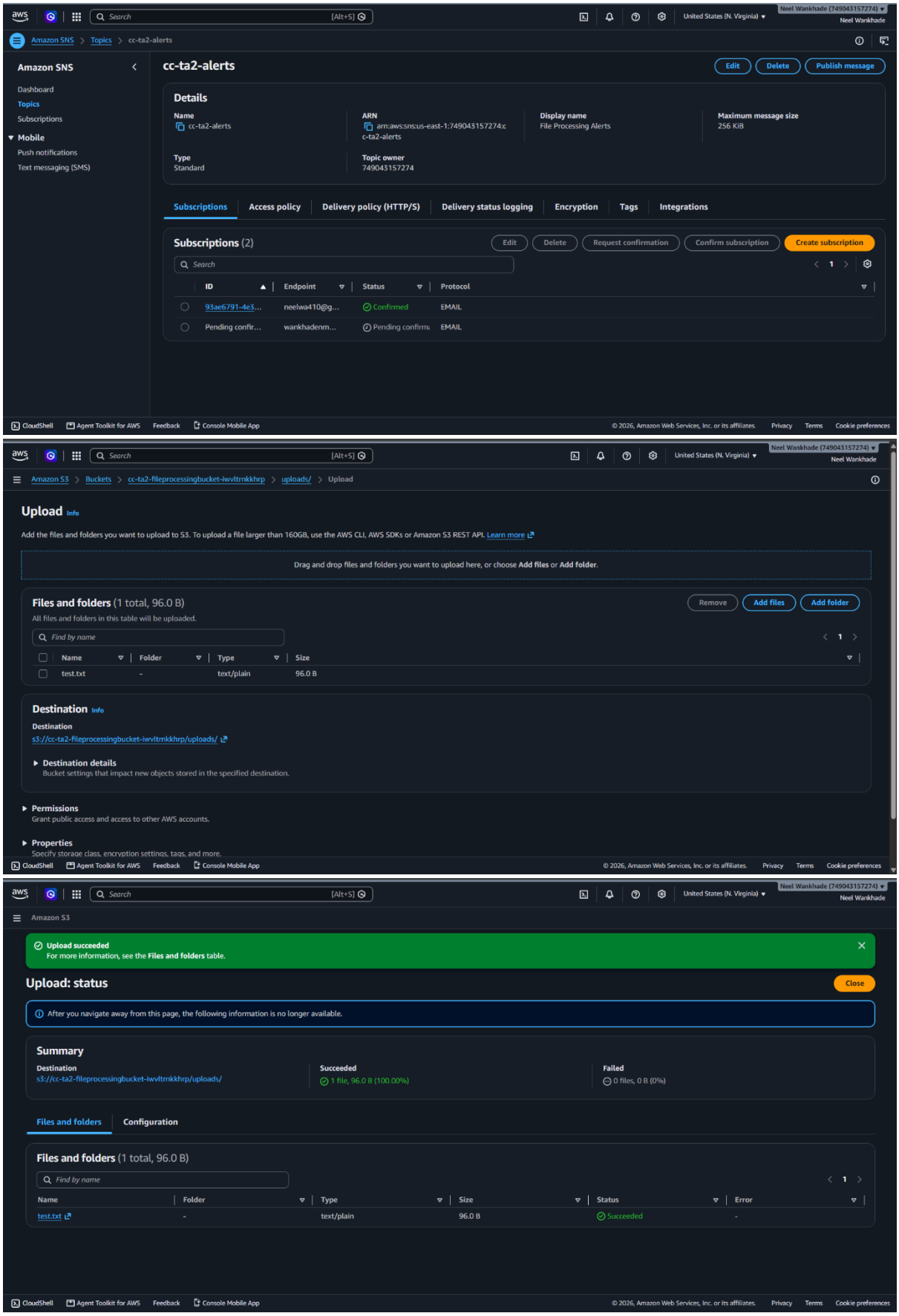
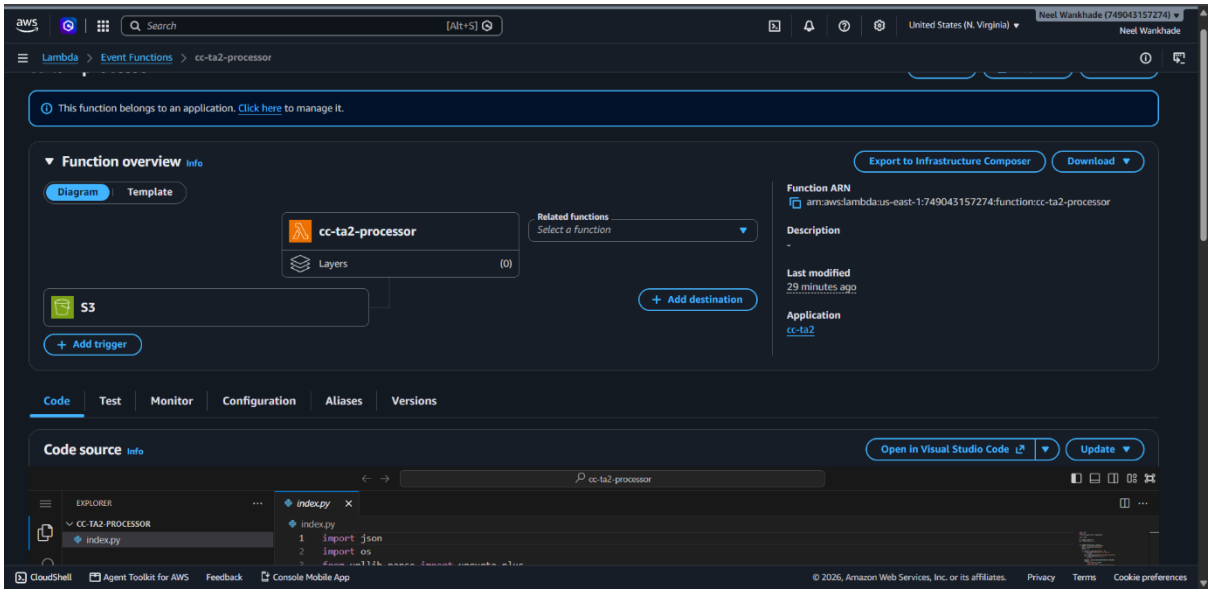
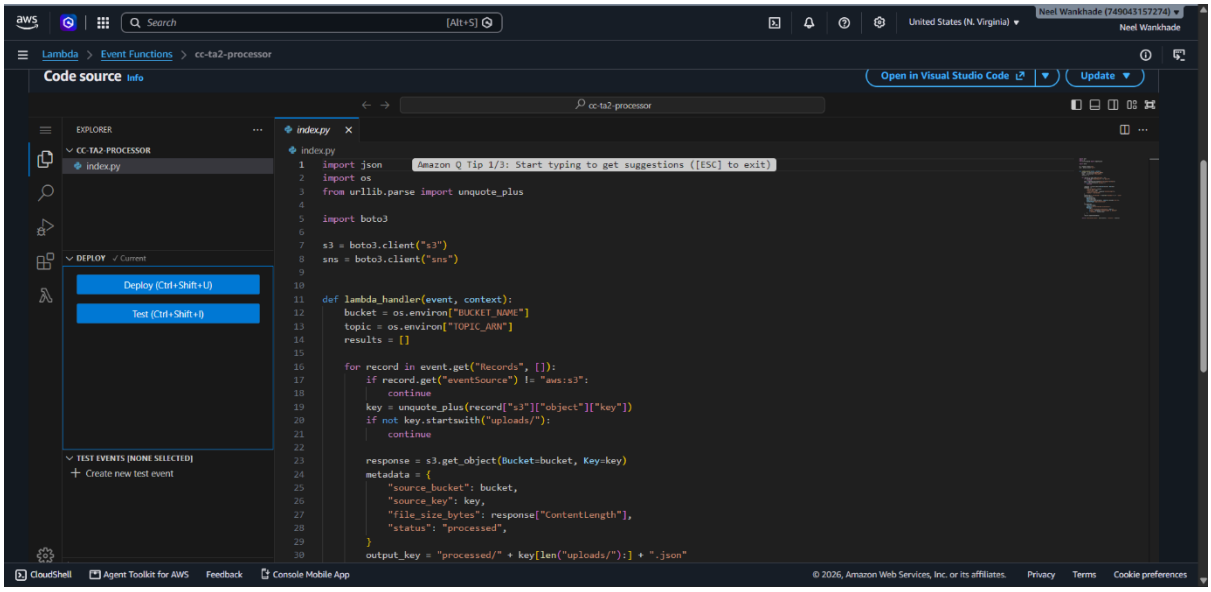


Figure 4: Test file uploaded to Amazon S3.

Step 4: Trigger the Lambda Function

After the test file was uploaded, the S3 event notification invoked the cc-ta2-processor Lambda function. The function retrieved the uploaded object and generated JSON metadata.





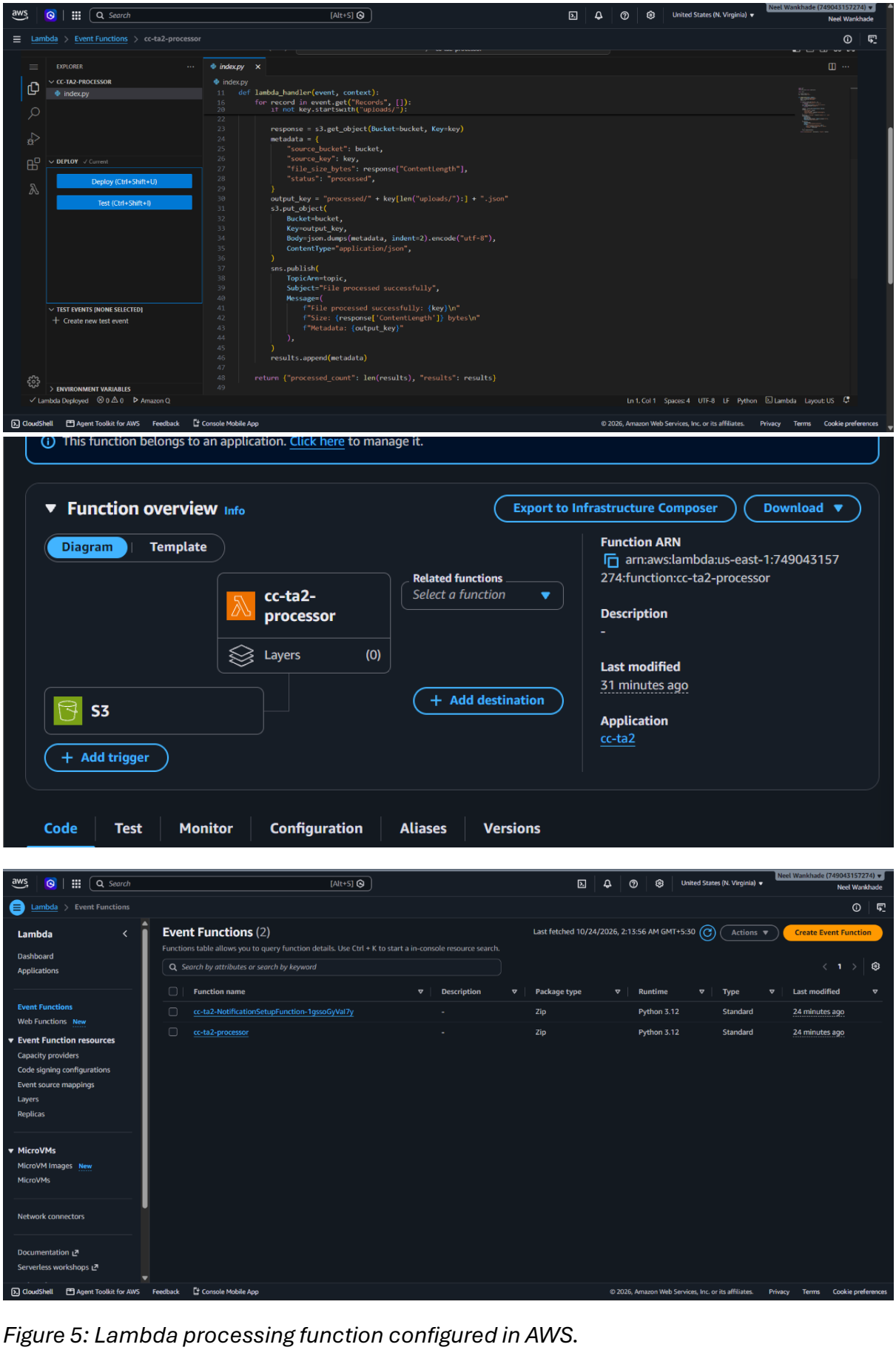


Figure 5: Lambda processing function configured in AWS.

Step 5: Verify Lambda Execution

Amazon CloudWatch Logs was used to inspect the Lambda function's execution. The log events were checked to verify that the function was invoked and to review its processing activity.

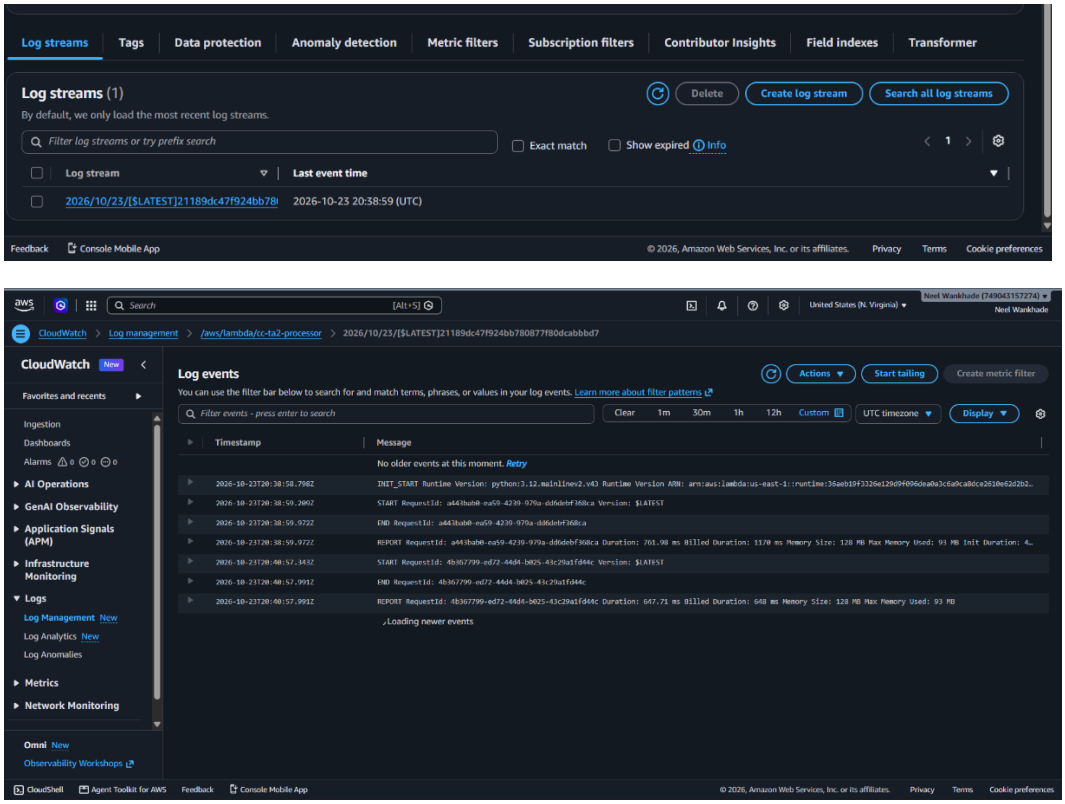
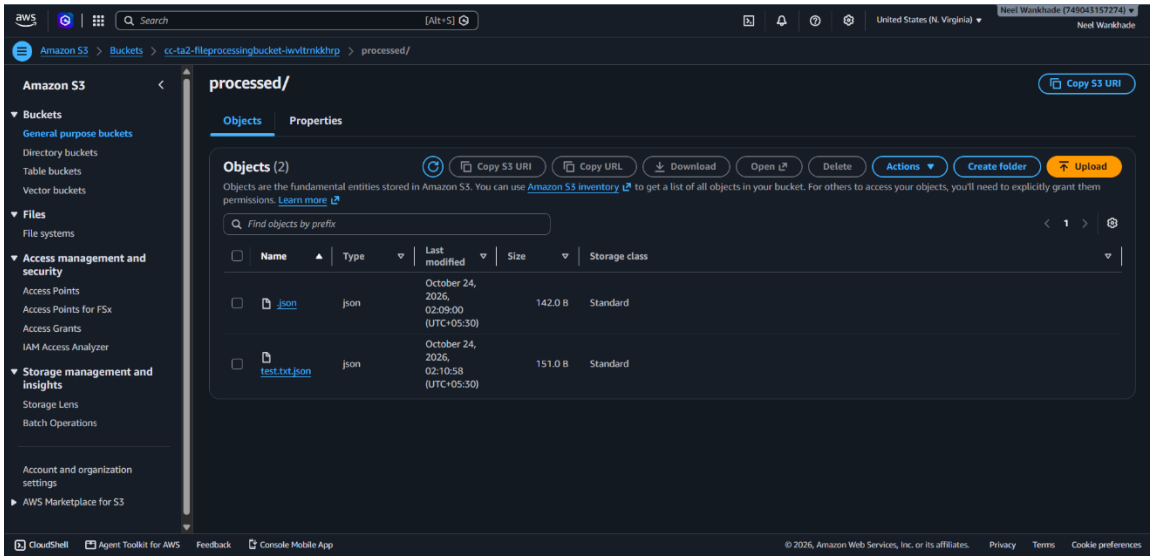
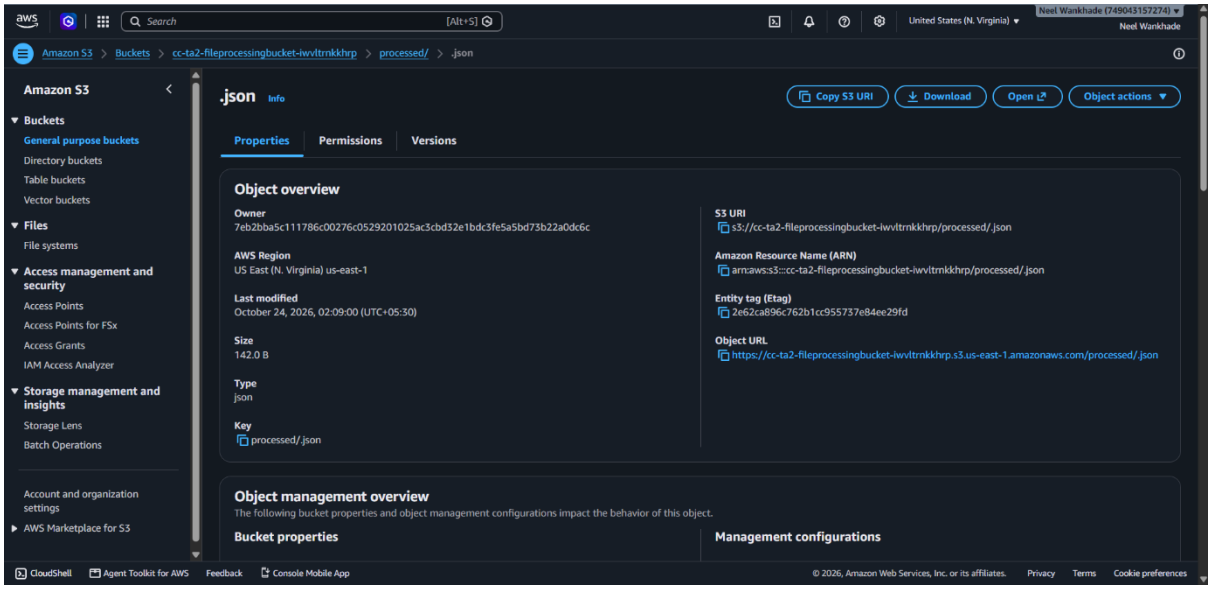


Figure 6: Execution logs of the Lambda processing function.

Step 6: Verify the Processed Output

The S3 bucket was opened again, and the processed/ prefix was inspected. The generated JSON metadata object was present after the test upload, confirming that the processing function had created the output.





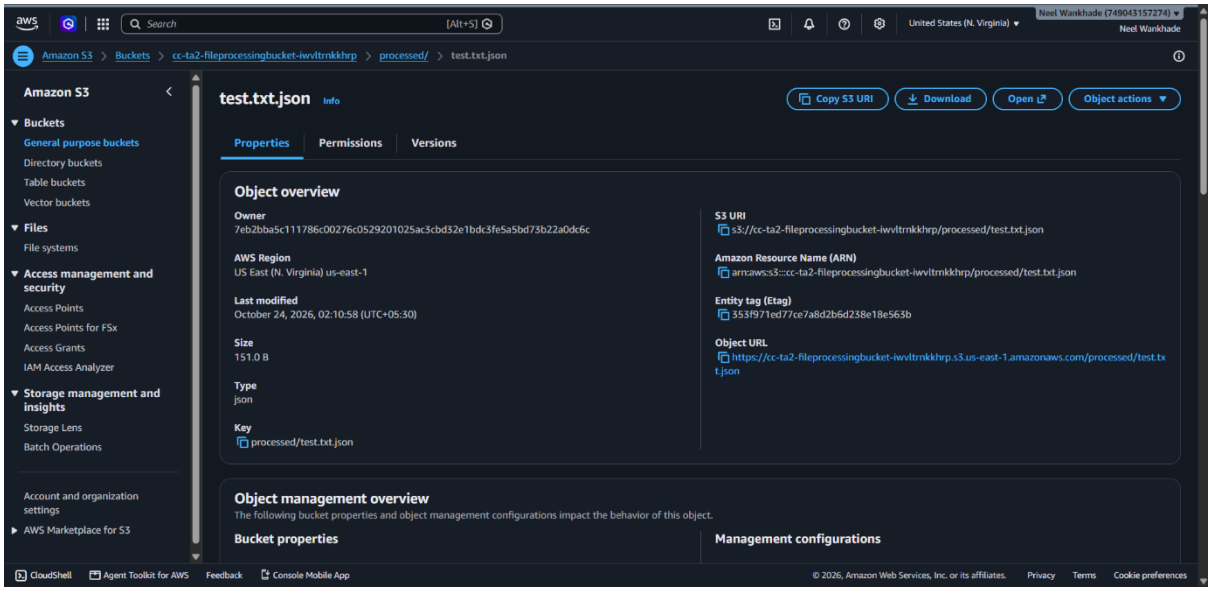


Figure 7: Generated JSON metadata stored in Amazon S3.

Step 7: Verify the SNS Email Notification

The selected email inbox was checked after processing. An email notification from the SNS-based file-processing workflow was received, confirming that the notification step had also completed successfully.

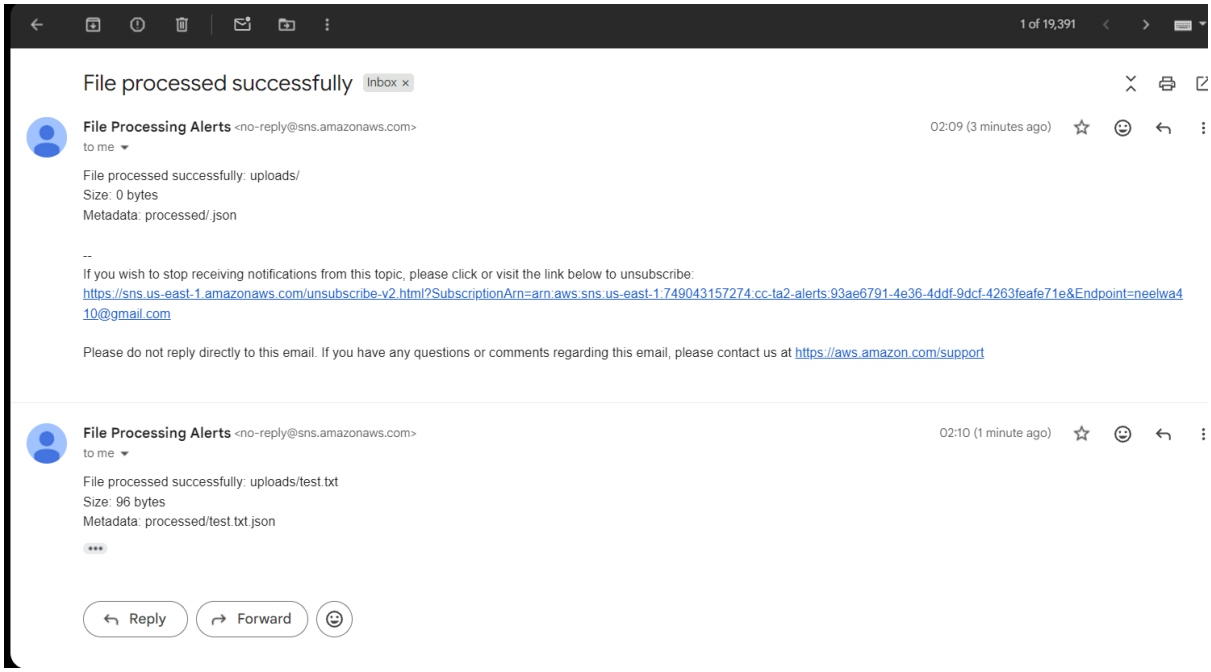


Figure 8: SNS email notification received after file processing.

7. Results and Observations

The event-driven file-processing workflow was successfully tested.

The following observations were made:

1. The CloudFormation stack reached the CREATE_COMPLETE status.
2. A test file was uploaded to the S3 uploads/ prefix.
3. The upload triggered the Lambda processing function.
4. The function generated JSON metadata and stored it under the processed/ prefix.
5. The Lambda execution could be inspected using CloudWatch Logs.
6. An SNS email notification was received at the confirmed email address.

These observations demonstrate that the main S3-to-Lambda-to-S3-and-SNS workflow operated successfully during testing.

8. Advantages

- **Automation:** File processing starts automatically when an object is uploaded.
- **Serverless execution:** Lambda runs the processing code without requiring a dedicated server.
- **Infrastructure as code:** CloudFormation provides a repeatable way to create and manage the AWS resources.
- **Loose coupling:** S3, Lambda, and SNS perform separate roles and communicate through events and messages.
- **Monitoring:** CloudWatch Logs supports inspection and troubleshooting of Lambda executions.
- **Security:** S3 public access blocking, server-side encryption, IAM permissions, and private-subnet placement help protect resources.

9. Conclusion

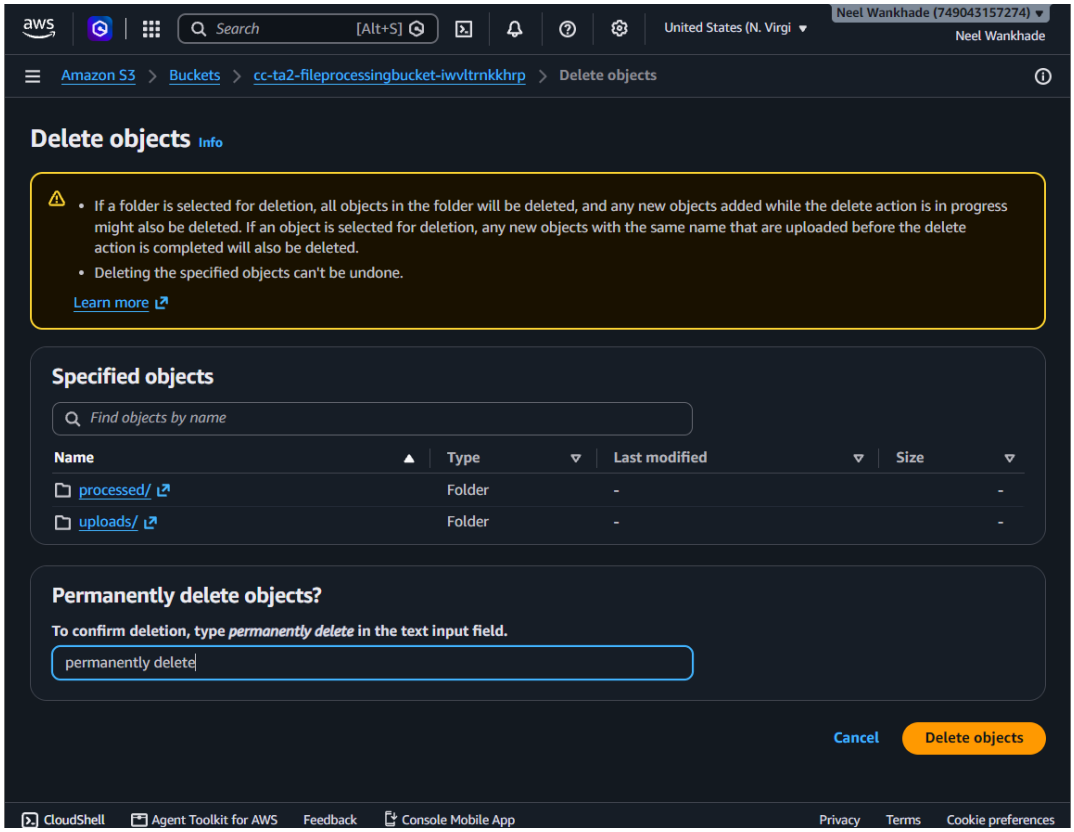
The practical demonstrated the design and deployment of an event-driven serverless file-processing architecture using AWS CloudFormation. Amazon S3 served as the file storage and event source, AWS Lambda processed uploaded objects and generated JSON metadata, and Amazon SNS delivered email notifications.

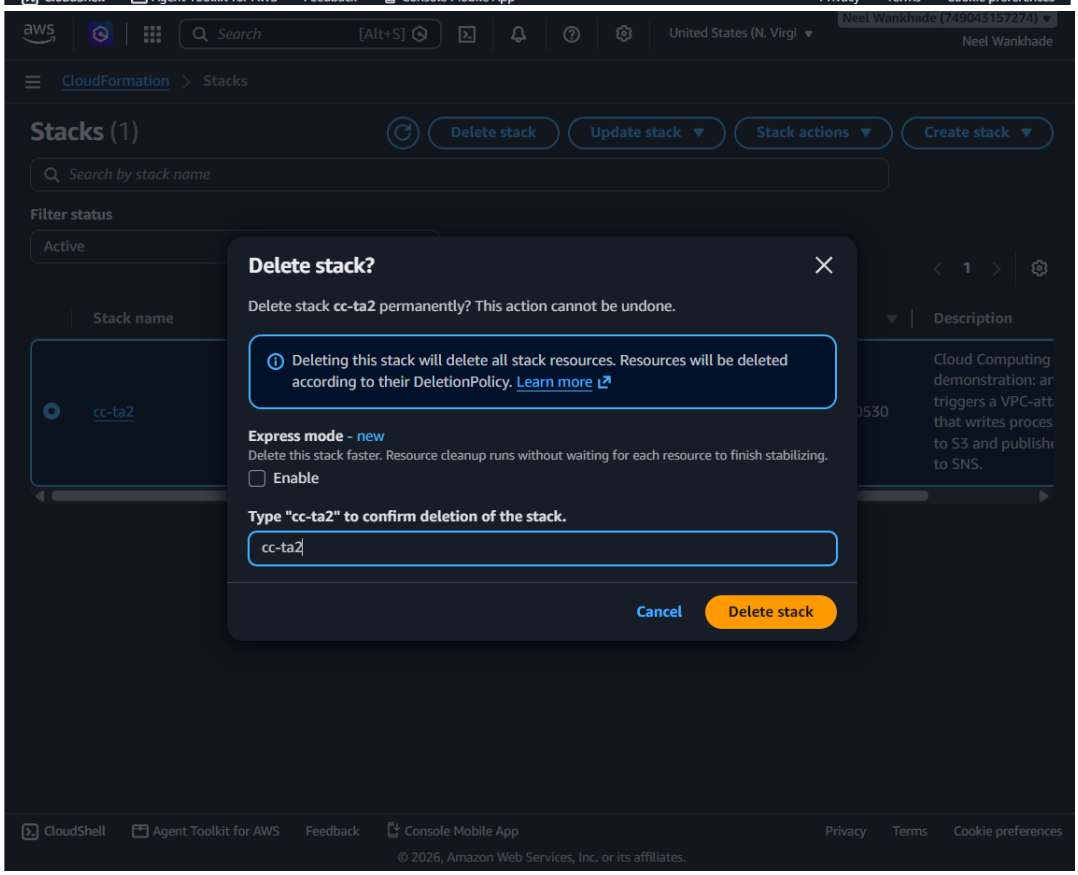
The successful test upload, generated JSON output, Lambda execution evidence, and received email notification confirmed that the principal workflow was functioning as intended.

10. Cleanup

After collecting the required evidence, deletion of the cc-ta2 CloudFormation stack was initiated to reduce the possibility of ongoing AWS charges, particularly from the NAT Gateway.

Note: The stack deletion must be verified separately in CloudFormation. The cleanup process should only be considered complete after the stack reaches DELETE_COMPLETE and any leftover resources have been checked.


Search [Alt+S]
United States (N. Virgi
Neel Wankhade (749043157274)
Neel Wankhade
Amazon S3 > Buckets > cc-ta2-fileprocessingbucket-iwvltrnkkhrp > Delete objects
Delete objects Info
If a folder is selected for deletion, all objects in the folder will be deleted, and any new objects added while the delete action is in progress might also be deleted. If an object is selected for deletion, any new objects with the same name that are uploaded before the delete action is completed will also be deleted.
Deleting the specified objects can't be undone.
Learn more
Specified objects
Find objects by name
Name
Type
Last modified
Size
processed/
Folder
-
uploads/
Folder
-
Permanently delete objects?
To confirm deletion, type permanently delete in the text input field.
permanently delete
Cancel
Delete objects
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
Privacy
Terms
Cookie preferences


Search [Alt+S]
United States (N. Virgi
Neel Wankhade (749043157274)
Neel Wankhade
CloudFormation > Stacks
Stacks (1)
Delete stack
Update stack
Stack actions
Create stack
Search by stack name
Filter status
Active
Stack name
Description
cc-ta2
Delete stack?
Delete stack cc-ta2 permanently? This action cannot be undone.
Deleting this stack will delete all stack resources. Resources will be deleted according to their DeletionPolicy. Learn more
Express mode - new
Delete this stack faster. Resource cleanup runs without waiting for each resource to finish stabilizing.
Enable
Type "cc-ta2" to confirm deletion of the stack.
cc-ta2
Cancel
Delete stack
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
Privacy
Terms
Cookie preferences
© 2026, Amazon Web Services, Inc. or its affiliates.